Project (3). Arduino Temperature Data Logger — 15 points

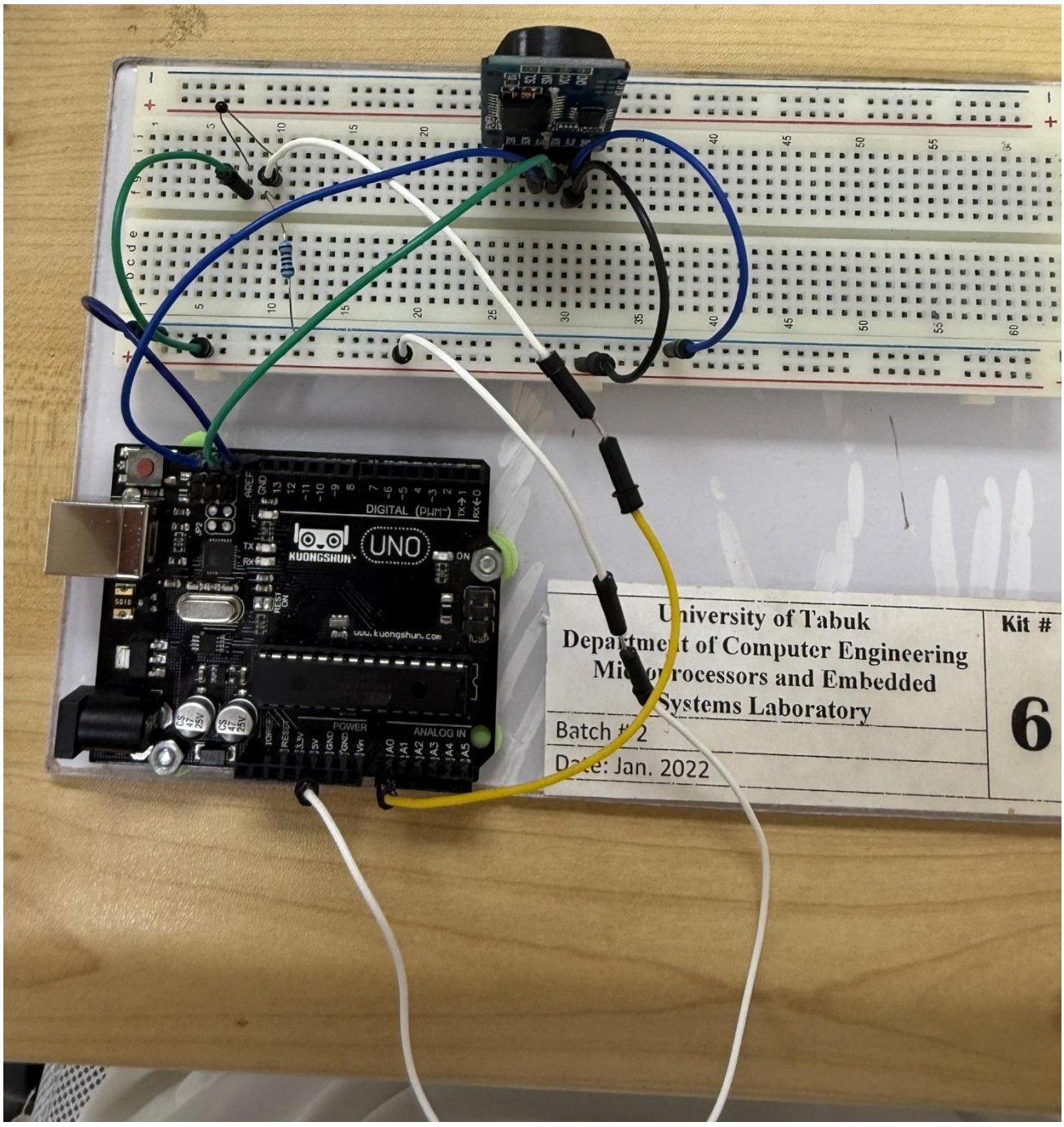
1. Project description

This project focuses on building a **temperature monitoring and logging system** using an **Arduino UNO**, a **thermistor sensor**, and a **DS3231 Real Time Clock (RTC) module**.
The system measures the surrounding temperature and records **one reading every 30 minutes**, precisely when the minute value equals **00** or **30**.

The thermistor detects the temperature and converts it into an analog signal, which is read by the Arduino through an analog input pin.
Using the **Beta equation**, the Arduino calculates the corresponding temperature in degrees Celsius.
The **DS3231 RTC module** provides accurate real-time information, allowing each reading to be associated with an exact date and time.

All readings are displayed on the **Serial Monitor** and simultaneously captured using the **CoolTerm** application.
CoolTerm is a serial communication tool that allows the user to **record and save the Arduino's serial data** directly to a text file on the computer.
This enables the temperature readings to be **stored, reviewed, and analyzed** later for observation and documentation purposes.

2. Project schematic Diagram



3. Algorithm and Program source code

Algorithm:

1. Start the system and initialize the Arduino UNO, thermistor sensor, and DS3231 RTC module.
2. Continuously read the current time from the DS3231 module.
3. Check if the current minute equals 00 or 30:
    - If true, take one temperature reading.
4. Read the analog voltage from the thermistor using the Arduino's analog input pin.
5. Calculate the resistance of the thermistor using the voltage divider formula.
6. Apply the Beta equation to convert resistance into temperature (°C).
7. Display the date, time, and temperature on the Serial Monitor.
8. Prevent duplicate readings within the same minute.
9. Wait for the next 30-minute interval and repeat the process.

Program Source Code:

```
#include <Wire.h>
#include "RTClib.h"

RTC_DS3231 rtc;

const int thermistorPin = A0;
const float BETA = 3950;
const float T0 = 298.15;
const float R0 = 10000;

bool alreadyRead = false;

void setup() {
  Serial.begin(9600);
  rtc.begin();
  if (rtc.lostPower()) {
    rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
  }
}

void loop() {
```

```
  DateTime now = rtc.now();
  int minuteNow = now.minute();
  int secondNow = now.second();

  if ((minuteNow == 0 || minuteNow == 30) && !alreadyRead &&
    secondNow < 10) {
   int analogValue = analogRead(thermistorPin);
   float resistance = 10000.0 * ((1023.0 / analogValue) - 1);
   float temperatureK = 1 / (1 / T0 + (1.0 / BETA) * log(resistance / R0));
   float temperatureC = temperatureK - 273.15;

   Serial.print(now.year());
   Serial.print("/");
   Serial.print(now.month());
   Serial.print("/");
   Serial.print(now.day());
   Serial.print(" ");
   if (now.hour() < 10) Serial.print("0");
   Serial.print(now.hour());
   Serial.print(":");
   if (now.minute() < 10) Serial.print("0");
   Serial.print(now.minute());
   Serial.print(":");
   if (now.second() < 10) Serial.print("0");
   Serial.print(now.second());
   Serial.print(" - Temperature: ");
   Serial.print(temperatureC);
   Serial.println(" °C");

   alreadyRead = true;
  }

  if (minuteNow != 0 && minuteNow != 30 && alreadyRead) {
   alreadyRead = false;
  }

  delay(1000);
}
```

4. Program description

The program starts by initializing the **Arduino UNO**, the **DS3231 real-time clock (RTC)**, and the **thermistor** sensor.
After setup, the Arduino continuously checks the current time from the RTC module.
When the **minute value** equals **00** or **30**, and no reading has been taken during that time interval, the system takes **one temperature measurement**.

The thermistor provides an analog voltage that varies with temperature.
The Arduino reads this signal, calculates the corresponding **resistance**, and then applies the **Beta equation** to convert it into **temperature in Celsius**.
The program then displays the **current date, time, and temperature** on the Serial Monitor.

A control variable ensures that the system takes only **one reading every 30 minutes** and avoids duplicate outputs within the same interval.
The process repeats indefinitely, allowing continuous and accurate temperature monitoring synchronized with real time.